JobGuard: An End-to-End MLOps Pipeline for Fraudulent Job Posting Detection

Name: Pawar Devesh Pramod Roll No: ME22B176

1. Abstract

The rise of fake job postings is a serious issue for job seekers, often resulting in financial losses and exposed personal information. Spotting these scams is very hard due to extreme class imbalances, as fake listings usually make up less than 5% of total postings. JobGuard is a complete Machine Learning Operations (MLOps) platform designed to solve this issue reliably on a large scale. Instead of using a static model, JobGuard offers a fully containerized continuous training and monitoring process. The main detection engine uses a Gradient Boosting Classifier, trained on a memory-efficient mix of TF-IDF text vectors and categorical metadata. This data is securely versioned with DVC and MLflow. To tackle model decay, Apache Airflow manages daily monitoring for data drift using Population Stability Index (PSI) metrics. If drift is detected, the system automatically gathers user-validated feedback from a secure FastAPI backend and starts a retraining process. Importantly, there is a strict "Human-in-the-Loop" (HITL) deployment check that requires admin review before new model weights are promoted to production. With real-time monitoring from Prometheus and Grafana, JobGuard shows how modern MLOps practices can turn a standard NLP classification task into a strong, production-ready fraud detection tool.

2. Introduction & Problem Statement

The Rise of Employment Scams

As global recruitment shifts to digital job boards and automated applicant tracking systems, bad actors have taken advantage of these platforms to carry out complex employment scams. These fake listings aim to collect sensitive Personally Identifiable Information (PII), charge illegal "application fees," or involve victims in money-laundering schemes. As the amount of online job data increases, manual moderation has become unfeasible, making automated, machine-learning-driven detection systems necessary.

The Engineering Challenge

Creating a model to spot these postings is just a small part of the challenge; the real difficulty is making it work. First, the data is highly imbalanced—fraudulent jobs make up about 4.3% of standard datasets like EMSCAD. This requires careful stratification, penalty weighting, and precision-recall optimization to prevent the system from being overwhelmed by false positives.

Second, fraud is adversarial. Scammers constantly change their language to avoid detection, which causes static machine learning models to experience "concept drift." A model that achieves a 95% F1 score in Production will likely perform worse once it is used in the real world, as the underlying data distribution changes.

Project Objectives

The JobGuard project's goal was to go beyond initial model training and build a sustainable machine learning ecosystem. The main aims were to:

1. Create a highly effective, CPU-friendly classifier for spotting fraud through sparse text embeddings and metadata.
2. Set up automated monitoring to check inference time, API health, and data drift in real-time.
3. Develop a secure, privacy-first telemetry loop that captures user corrections and adds them back into the training dataset.
4. **Create a Continuous Training (CT) pipeline** that automates data versioning and model tracking while keeping a vital "Human-in-the-Loop" validation step for safety in deployment.

3. Data Methodology & Feature Engineering

3.1. Exploratory Data Analysis

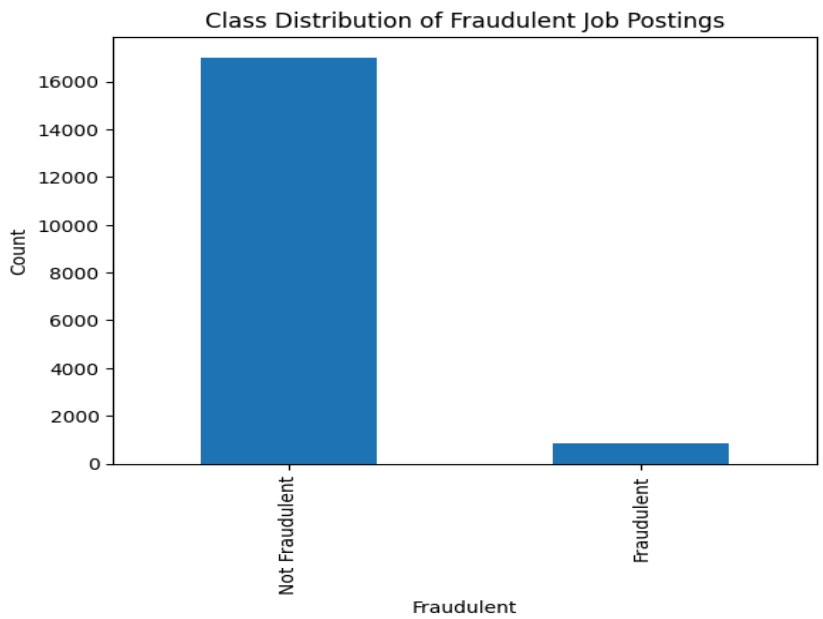
The EMSCAD dataset has been used for training and testing purposes. It consists of 18 columns; out of which 4 are numerical/binary (`telecommuting, has_company_logo, has_questions, fraudulent`) and others are categorical.

The dataset consists of 17880 rows with following null values distribution:

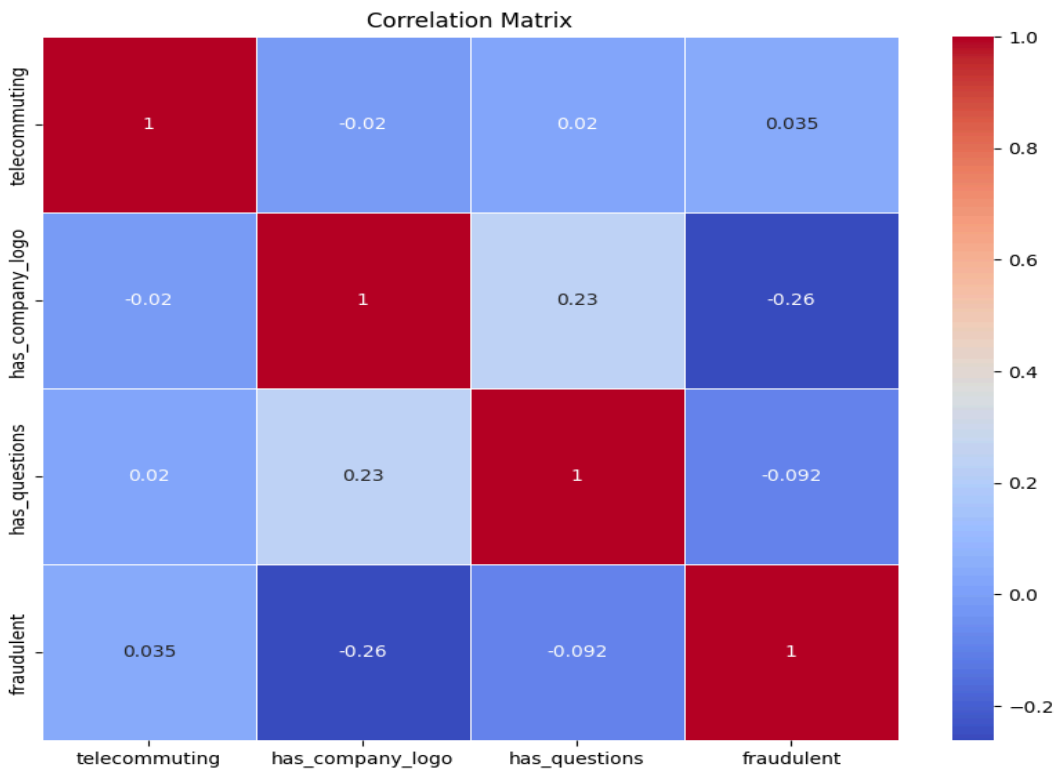
```
job_id                    0
```

title	0	
location	346	
department	11547	(dropped)
salary_range	15012	(dropped)
company_profile	3308	
description	1	
requirements	2696	
benefits	7212	
telecommuting	0	
has_company_logo	0	
has_questions	0	
employment_type	3471	
required_experience	7050	
required_education	8105	
industry	4903	
function	6455	
fraudulent	0	

The EMSCAD dataset has a huge class imbalance as shown below (~4.83% Fraudulent)



The correlation between the binary columns is also explored



This correlation matrix shows that there is no significant correlation between fraudulent columns with other binary columns suggesting the importance of other categorical variables is significant.

3.2. Dataset & Stratification

The system uses a dataset of real and fake job postings. Due to the large class imbalance; where fake listings are a small part of the data. We use an 80/20 train-test split that is carefully stratified against the fraudulent target variable. This ensures that the natural distribution of fraud is kept across all testing and training sets, preventing the model from artificially boosting its accuracy by just predicting the majority class.

3.2. Feature Extraction & Vectorization

Vectorization Instead of processing text and categorical data separately, our pipeline combines them into a single, memory-efficient mathematical format.

- **Text Processing:** Key text fields (title, location, description, company profile, requirements, and benefits) are combined into one text block. This consolidated text is transformed using a TF-IDF vectorizer, limited to 5,000 features and using a 1-2 n-gram range to capture local context.
- **Metadata Flags:** The pipeline also extracts 7 binary flags from the raw data, including indicators for `has_company_logo`, `has_questions`, `has_salary`, and `telecommuting`.
- **Matrix Assembly:** The text vectors and metadata are combined into a highly optimized `scipy.sparse.csr_matrix`. This keeps the data footprint light enough for rapid processing on CPU infrastructure. To avoid data leakage, we save the exact state of the TF-IDF vectorizer as an artifact for later transforming the validation and inference data.

3.3. Immutability via DVC

Every part of the data transformation, from splitting to preprocessing and baseline generation, is mapped and version-controlled with a `dvc.yaml` pipeline. This ensures that specific model weights can always be traced back to the exact snapshot of data they were trained on.

DVC dag

4. Model Development & Tracking

4.2. Core Algorithm & Penalty Weighting

We selected a `GradientBoostingClassifier` for the core detection engine. Gradient boosting is great for handling sparse, high-dimensional data and trains efficiently without needing GPU clusters. To deal with the limited number of fraudulent examples, we apply specific sample weighting during training, heavily penalizing the model for misclassifying a fraudulent job.

4.3. Experiment Tracking (MLflow)

The training engine integrates smoothly with an MLflow tracking server. During its operation, it logs important hyperparameters (e.g., `n_estimators`, `max_depth`, `learning_rate`) and captures precise evaluation metrics like validation F1, precision, recall, and ROC-AUC. Additionally, it automatically creates and saves a visual confusion matrix artifact, providing a comprehensive record for every model iteration.

The following table is the logging of some manual experiments runs of MLFlow and how hyper parameters affected the validation metrics ,

n_estimators	max_depth	learning_rate	val_F1_fraud	val_roc_auc
200	5	0.1	0.6805	0.9693
200	7	0.1	0.7973	0.9879
200	6	0.01	0.4623	0.9637
250	10	0.1	0.7804	0.9840
700	7	0.1	0.7940	0.9869

5. System Architecture & Microservices

To ensure consistent environments across development and production, the entire JobGuard platform is containerized using Docker Compose. The structure separates concerns into distinct microservices:

- **Storage Layer:** A PostgreSQL database manages the backend state for both MLflow and Airflow.
- **Tracking & Orchestration:** Separate containers run the MLflow tracking server (port 5000) and the Apache Airflow orchestrator.
- **Observability:** A Prometheus time-series database collects system metrics, displayed via Grafana, with an Alertmanager service managing incident routing.
- **Inference:** A dedicated FastAPI container processes all incoming predictions and telemetry.

6. Production Serving & Security

6.1. The API Gateway & Dynamic Routing

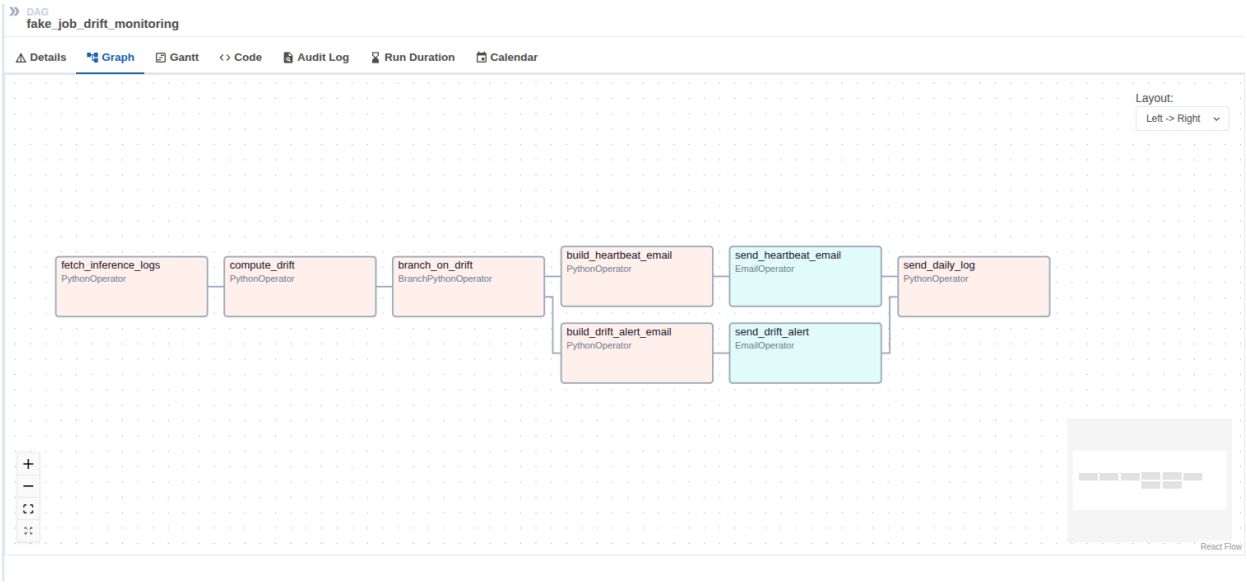
The FastAPI backend serves as the secure entry point for the system, offering key endpoints like `/predict` for live predictions and `/feedback` for user corrections. We designed the API with a dynamic `MODEL_SERVE_MODE` toggle. Depending on the environment variables, the API routes inference requests to either a locally cached memory object or over the Docker network to a dedicated MLflow HTTP serving container on port 5001.

6.2. Data Privacy & Encryption

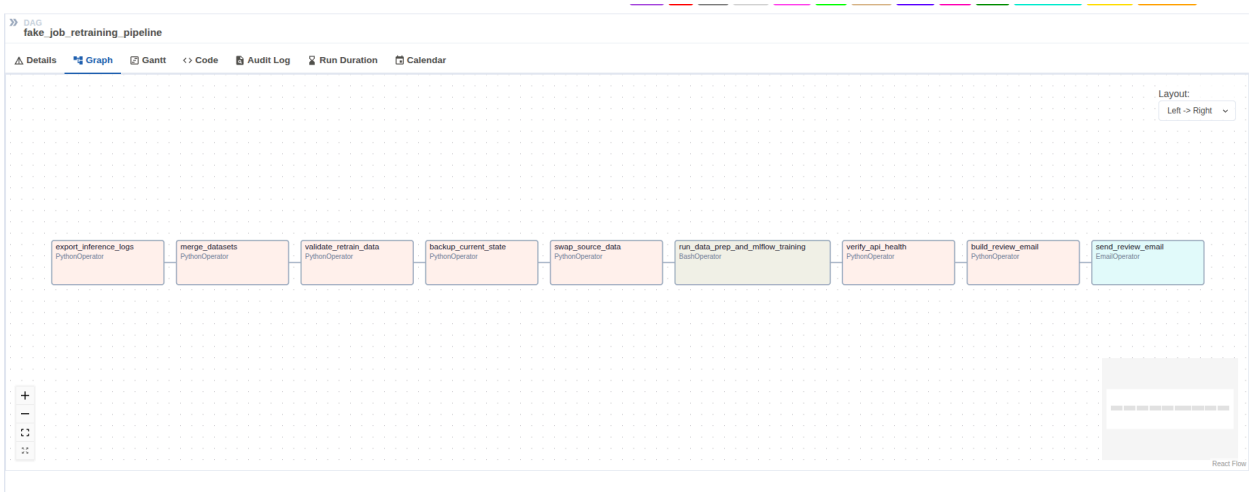
As the system processes job descriptions and company profiles, protecting Personally Identifiable Information (PII) is crucial. Before any prediction or user feedback is stored in the SQLite `inference.db` database, the API encrypts the data using a Fernet cipher. The main encryption key (`AIRFLOW__CORE__FERNET_KEY`) is securely injected at runtime, ensuring compliance with data privacy standards while allowing authorized retraining pipelines to decrypt the data later.

7. Orchestration & Continuous Training

Apache Airflow acts as the automated brain, connecting the data engineering and model training layers. When a retraining cycle starts, Airflow retrieves the decrypted, user-validated logs from the SQLite database and integrates them with the root DVC dataset. It then triggers the `dvc.yaml` pipeline, automatically running the split, preprocess, and featurize_test stages. Notably, the system follows a "**Human-in-the-Loop**" (HITL) deployment model. Once the new model finishes training and logs its metrics to MLflow, Airflow pauses the automated deployment. It sends an email alert to the engineering team with the new validation scores, requiring a human admin to manually review the artifacts and assign the `@production` alias in the MLflow UI before the model goes live.



Drift Detection Dag



Retraining Dag

8. Observability & Drift Detection

8.1. Live Telemetry

The model's behavior is monitored via a `/metrics` endpoint on the FastAPI backend. Prometheus continuously scrapes custom histograms and gauges, tracking key health indicators like `api_request_latency_seconds`, `model_fraud_rate_current`, and raw `model_prediction_confidence`.

8.2. Proactive Alerting

We set up strict PromQL rules in Alertmanager to quickly identify system problems.

8.2.1 API Health Alerts

- **HighErrorRate (Critical)**
  - **Trigger:** API error rate exceeds 5% over a 5-minute window.
  - **Action:** Exceeds project SLA guidelines. Requires immediate investigation of the `/predict` endpoint logs.
- **APIDown (Critical)**
  - **Trigger:** FastAPI service is unreachable for 1 minute.
  - **Action:** Core service failure. Check Docker container logs (`docker logs fake-job-postings-detector-api-1`).
- **HighInferenceLatency (Warning)**

    - **Trigger:** The 95th percentile (p95) inference latency exceeds 200ms for 5 minutes.
    - **Action:** Breaches the 200ms latency SLA. Indicates a potential bottleneck in the MLflow serving gateway or API preprocessing.
- **SLABreachRateHigh (Warning)**
    - **Trigger:** More than 0.1 SLA breaches per second over 10 minutes.
    - **Action:** Consistent failure to meet latency targets, requiring performance profiling.

8.2.2 Data Drift Alerts

Monitors the statistical properties of incoming data against the training baseline to detect distribution shifts.

- **DataDriftDetected (Warning)**
    - **Trigger:** Population Stability Index (PSI) score > 0.2 on any feature for 5 minutes.
    - **Action:** Early warning of data distribution shift. Indicates that incoming data profiles are changing.
- **DriftConfirmed (Critical)**
    - **Trigger:** Drift flag remains active for 10 continuous minutes.
    - **Action:** Model reliability is compromised. Action required: Trigger the `fake_job_retraining_pipeline` DAG in Airflow to update the model.
- **FraudRateAnomaly (Warning)**
    - **Trigger:** Current fraud prediction rate deviates by more than 15% from the ~4.8% training baseline for 10 minutes.
    - **Action:** Indicates severe concept drift, a change in attacker behavior, or a data pipeline failure.

8.2.3 Model Performance Alerts

Evaluates the confidence and utilization of the deployed ML model.

- **LowPredictionConfidence (Warning)**
    - **Trigger:** Median model confidence score drops below 60% for 10 minutes.
    - **Action:** The model is struggling to classify recent inputs, strongly suggesting the need for retraining on newer, more representative data.
- **NoRecentPredictions (Info)**
    - **Trigger:** Zero predictions made in the last 30 minutes.
    - **Action:** May indicate natural idle periods or an upstream failure in the frontend preventing traffic from reaching the API.

8.2.4 Infrastructure Alerts

Ensures the monitoring stack itself remains fully operational.

- **PrometheusDown (Critical)**
  - **Trigger:** Prometheus service is unreachable for 1 minute.
  - **Action:** Monitoring backbone failure. Alertmanager cannot function without it.
- **GrafanaDown (Warning)**
  - **Trigger:** Grafana dashboard service is unreachable for 2 minutes.
  - **Action:** Visualization layer is down; raw metrics are still collecting but dashboards are temporarily unavailable.

9. Challenges

1. Container Boot Timing and Healthcheck Synchronization

- **Challenge:** The MLflow Model Serving container (Uvicorn) experienced extended boot times (up to 77 seconds) due to the overhead of loading the `GradientBoostingClassifier` into RAM. Docker's default healthchecks were failing before the model was fully loaded, resulting in infinite crash-and-restart loops. Furthermore, different MLflow components required different healthcheck endpoints (`/` or `/version` for the tracking server vs. strictly `/ping` for the scoring server).
- **Resolution:** Configured precise endpoint targeting for each container and implemented a generous `start_period: 120s` in `docker-compose.yml` to grant the containers a grace period for memory allocation without consuming retry attempts.

2. The Artifact Proxy vs. Golden Volume Mount Dilemma

- **Challenge:** When the automated Airflow DAG retrained the model, it registered the model using MLflow's modern HTTP proxy URI (`mlflow-artifacts:/`). The serving container attempted to download the weights over the network, but the tracking server failed to serve the files, resulting in fatal `OSError: No such file` crashes during boot.
- **Resolution:** Pivoted to a "Golden Volume Mount" architecture. Stripped out the HTTP proxy flags and mounted a single, shared absolute path (`./mlflow_artifacts`) directly into the Airflow, MLflow Tracking, and MLflow Model containers. This allowed Airflow to write directly to the disk and the serving container to instantly read from it, bypassing network latency and proxy bugs.

3. Airflow Orchestration and Resource Constraints

- **Challenge:** Transitioning Airflow to a local development machine introduced CPU/RAM constraints, causing the Gunicorn webserver workers to breach the default 120-second startup deadline and crash. Additionally, orchestrating the database migration, user creation, scheduler, and webserver within a single Docker entrypoint resulted in bash syntax and foreground-blocking errors.
- **Resolution:** Increased the `AIRFLOW__WEBSERVER__WEB_SERVER_MASTER_TIMEOUT` to 300 seconds. Refactored the entrypoint command to execute within a formal shell (`bash -c`) and forced the scheduler into a background process (`&`) so both core components could run concurrently.

4. Strict Type Enforcement in the FastAPI Gateway

- **Challenge:** The implementation of the Human-in-the-Loop feedback mechanism resulted in `422 Unprocessable Entity` errors. The frontend was sending natural language labels (e.g., "Legitimate"), which clashed with the FastAPI backend's rigid Pydantic schemas using strict Python `Literal["correct", "incorrect"]` typing.
- **Resolution:** Created a mapping logic layer in the frontend JavaScript to dynamically compare user actions against model predictions, translating them into the exact literal strings expected by the backend before executing the HTTP request.

5. MLflow Cross-Origin Resource Sharing (CORS) Blocks

- **Challenge:** The newly updated MLflow Tracking UI failed to load experiments or datasets, throwing `403 Forbidden` errors in the network logs despite the container being perfectly healthy. This was triggered by MLflow's strict FastAPI security middleware flagging `0.0.0.0` as an invalid HTTP origin header.
- **Resolution:** Standardized access protocols to enforce routing strictly through `http://localhost:5000` or `127.0.0.1`, satisfying the middleware's origin requirements without requiring custom proxy configurations.

6. Dynamic Model Routing and Deployment

- **Challenge:** Ensuring that a newly trained model from the Airflow DAG wasn't immediately pushed to users without human verification, while still allowing the system to serve traffic.
- **Resolution:** Developed a dual-mode FastAPI gateway controlled by a `MODEL_SERVE_MODE` environment variable. Integrated MLflow's alias system (`@production`), ensuring the serving container only pulls weights verified by an administrator, solidifying a safe CI/CD deployment pipeline.

10. Conclusion

The project shows that finding fake job postings needs more than just a precise notebook. By using DVC for unchangeable data tracking, MLflow for tracking experiments, and Airflow for organized, controlled continuous training, we have created a self-repairing ML system that can protect users and adjust to changes in tricky data in production.

11. AI Disclosure Appendix

1. The documentation formatting and README formatting was assisted by Google Gemini.
2. The api frontend, html emails, pytests and boilerplate codes for airflow dags are developed using Google Gemini.
3. Google Gemini is used to debug errors and bugs encountered during production.